Boundary Value Analysis

DAT321, DIT843

25/11/2025

Prepared by: Sabina Akbarova

sabina.akbarova@chalmers.se

Images sourced from: link, link

If you learn one thing today… Learn these

1. A boundary is a pair of inputs.
2. Validity groups are three high-level boundary categories.
3. Program derivative measures quality of boundaries (+formula).

If you learn one thing today… Learn these

1. A boundary is a pair of inputs.
2. Validity groups are three high-level boundary categories.
3. Program derivative measures quality of boundaries (+formula).

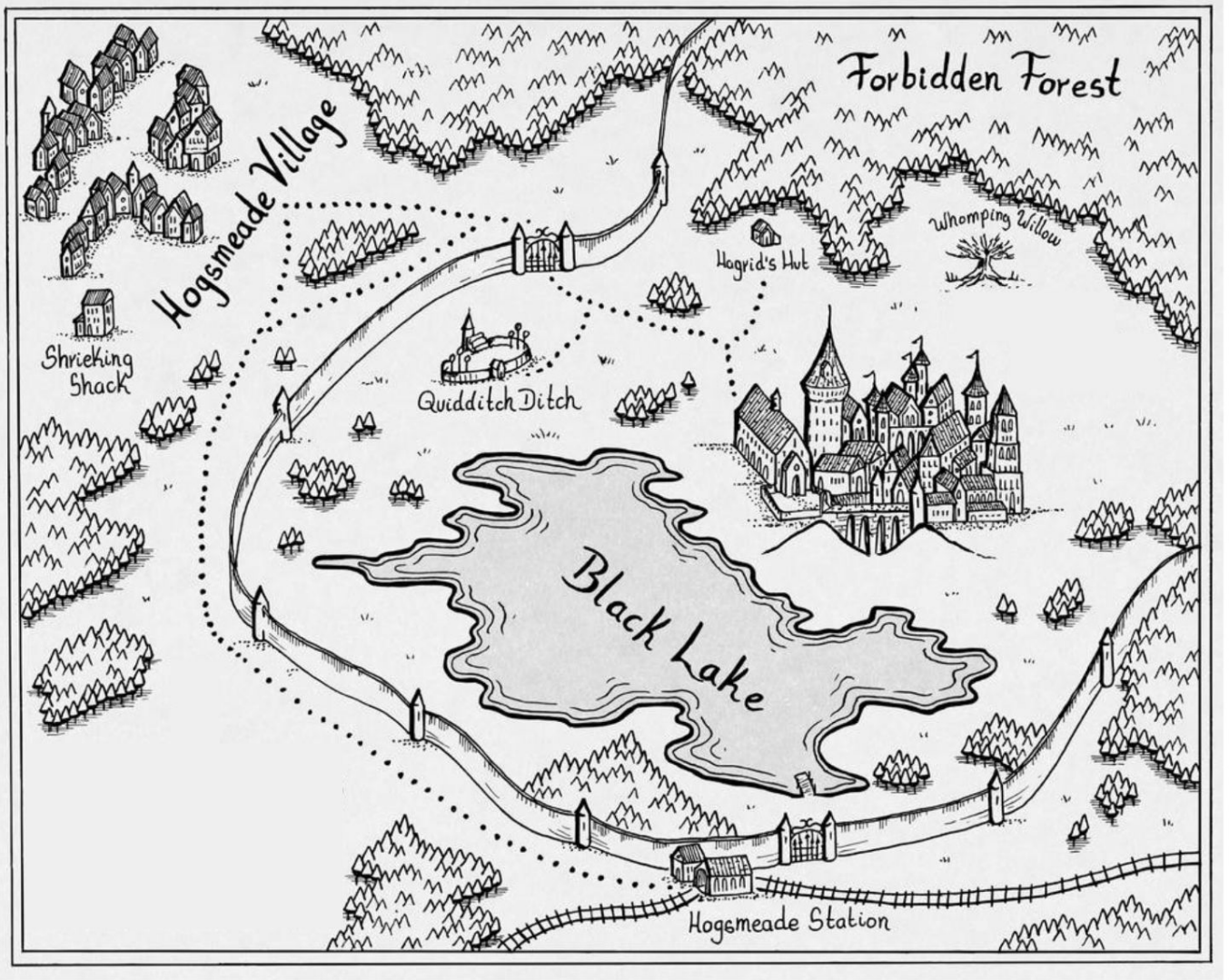
If you pay close attention, you'll also learn how many owls are hidden in these slides.

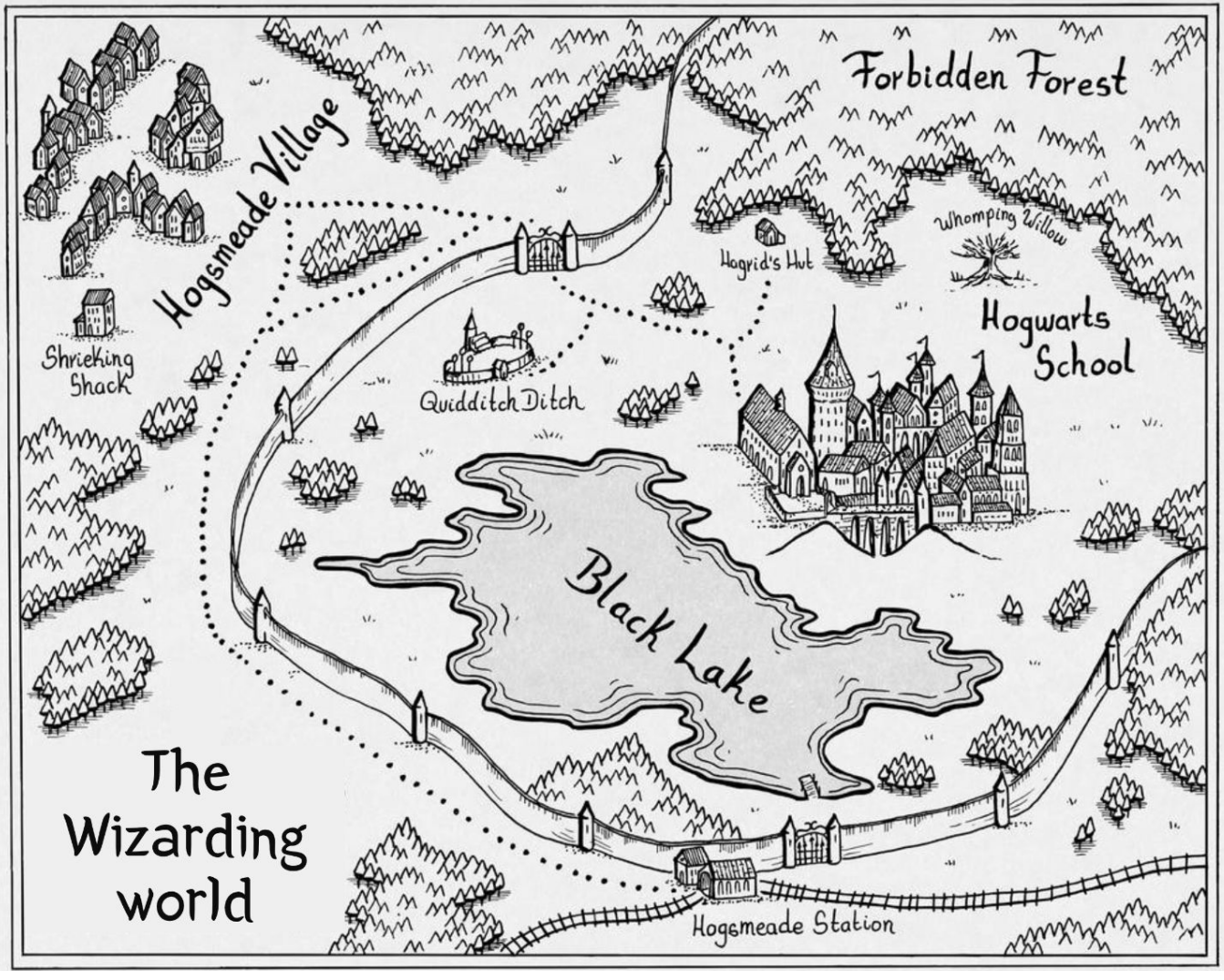
If you learn one thing today… Learn these

1. A boundary is a pair of inputs.
2. Validity groups are three high-level boundary categories.
3. Program derivative measures quality of boundaries (+formula).

If you pay close attention, you'll also learn how many owls are hidden in these slides.

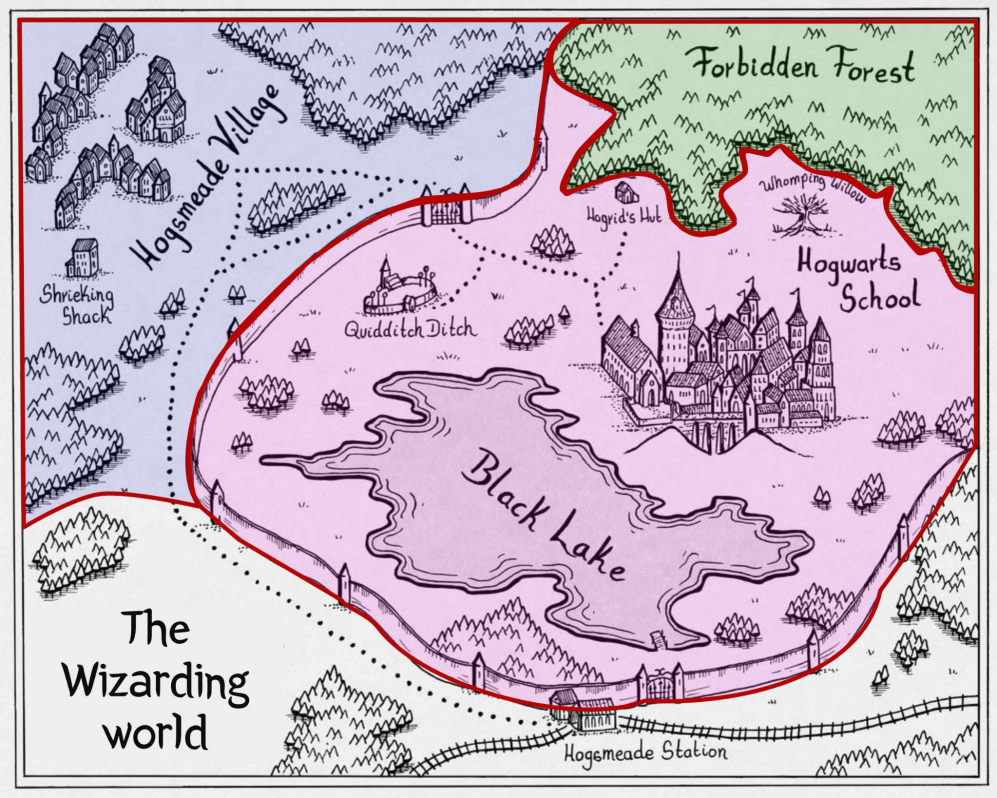
```
owl_count = 1
```


Forbidden Forest
Hogsmeade Village
Whomping Willow
Hagrid's Hut
Shrieking Shack
Quidditch Ditch
Black Lake
Hogsmeade Station

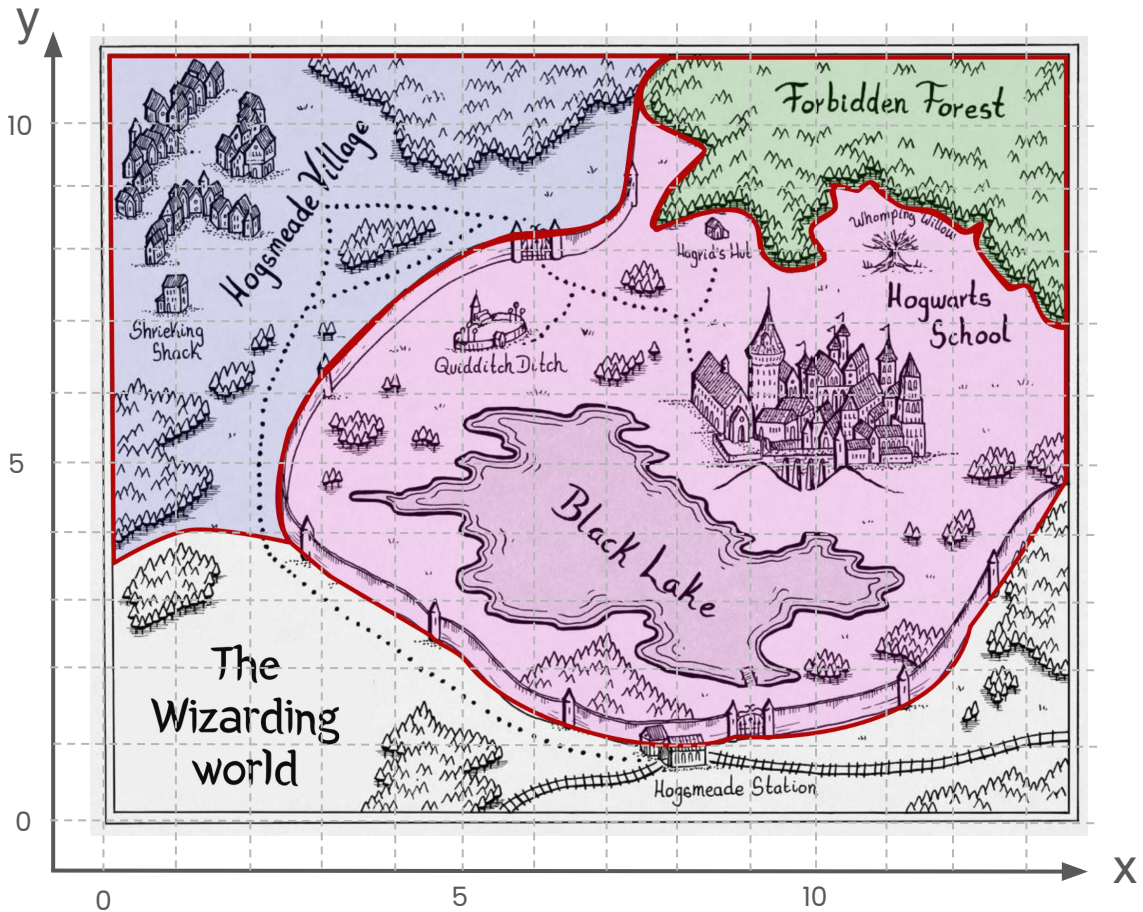


The Wizarding world

Equivalence partitioning



Equivalence partitioning

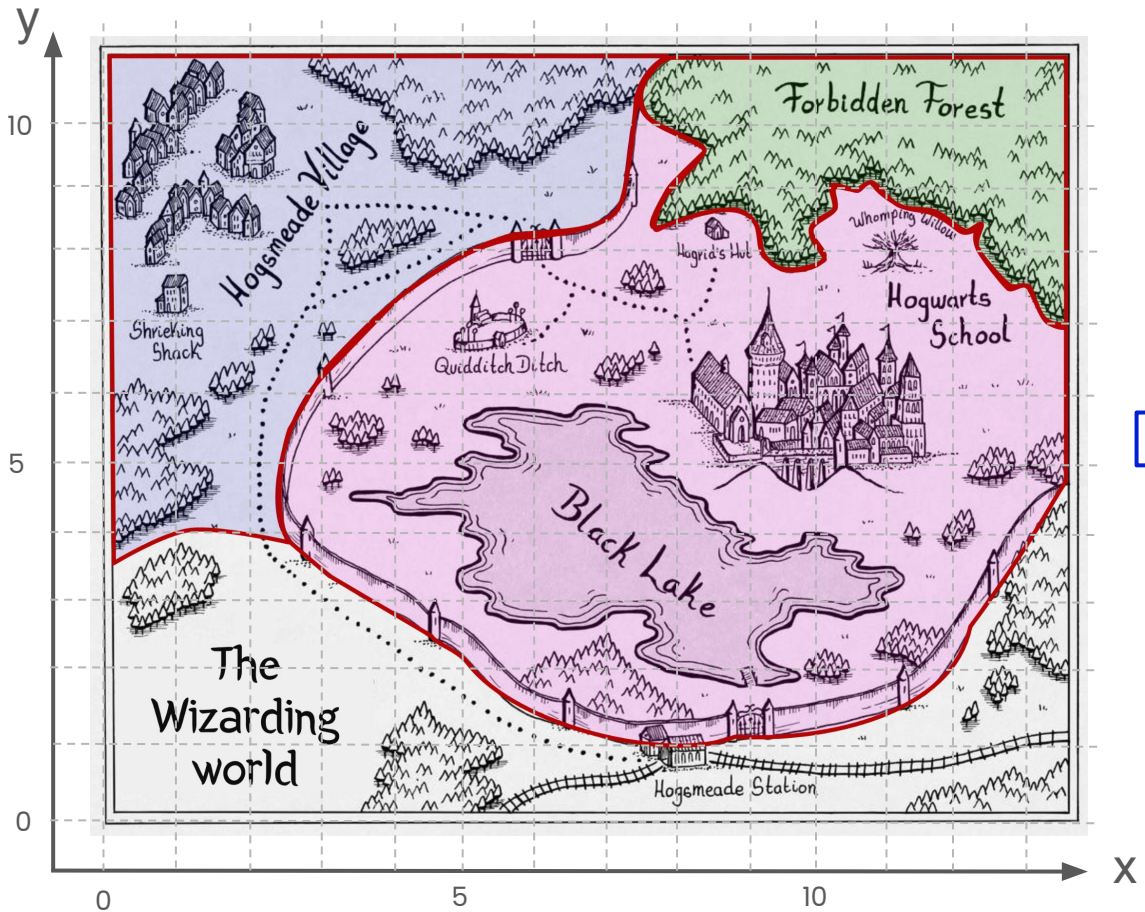


Example:

`locate_character(x, y)`

returns the name of the area corresponding to the coordinates.

Equivalence partitioning



input

Example:

`locate_character(x, y)`

returns the name of the area corresponding to the coordinates.

behaviour

Equivalence partitioning

The Wizarding World = Input space

input

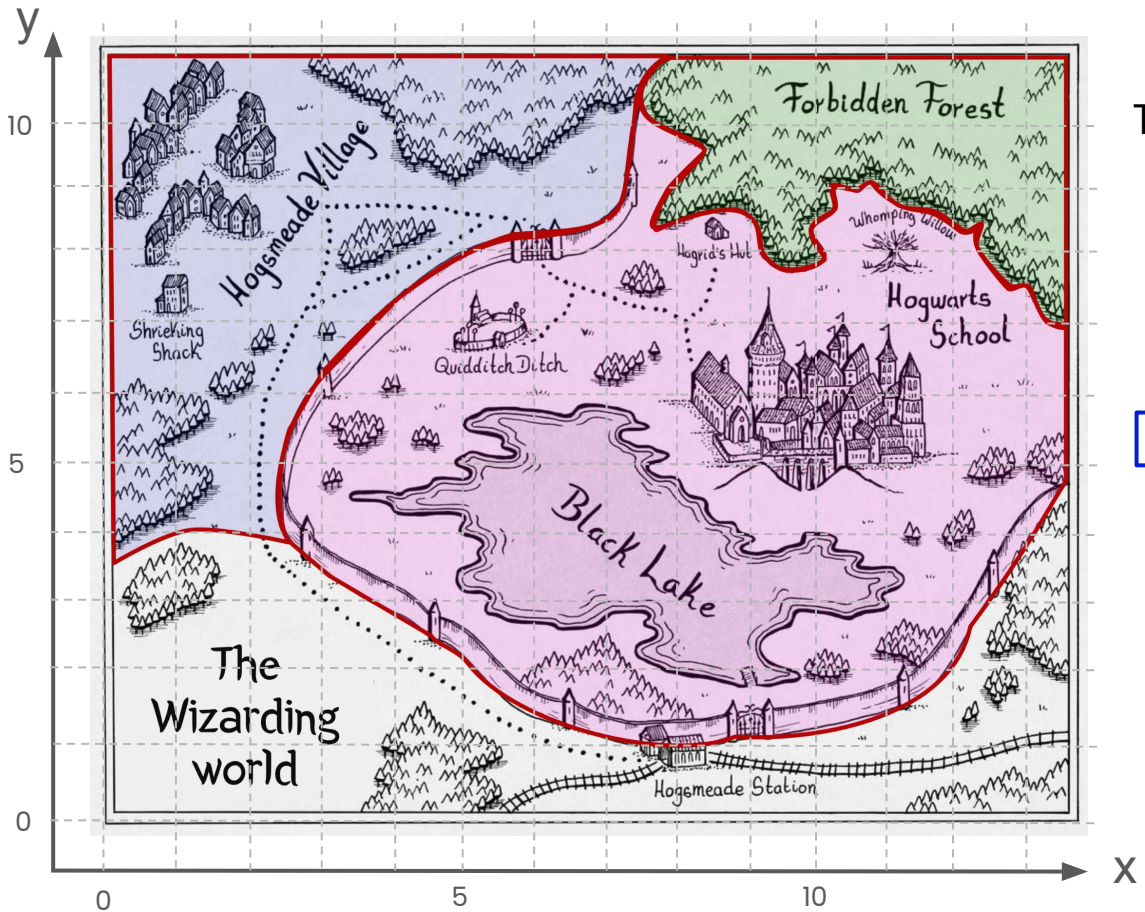
Example:

`locate_character(x, y)`

returns the name of the area corresponding to the coordinates.

behaviour

Equivalence partitioning



The Wizarding World = Input space

input

Example:

`locate_character(x, y)`

returns the name of the area corresponding to the coordinates.

behaviour

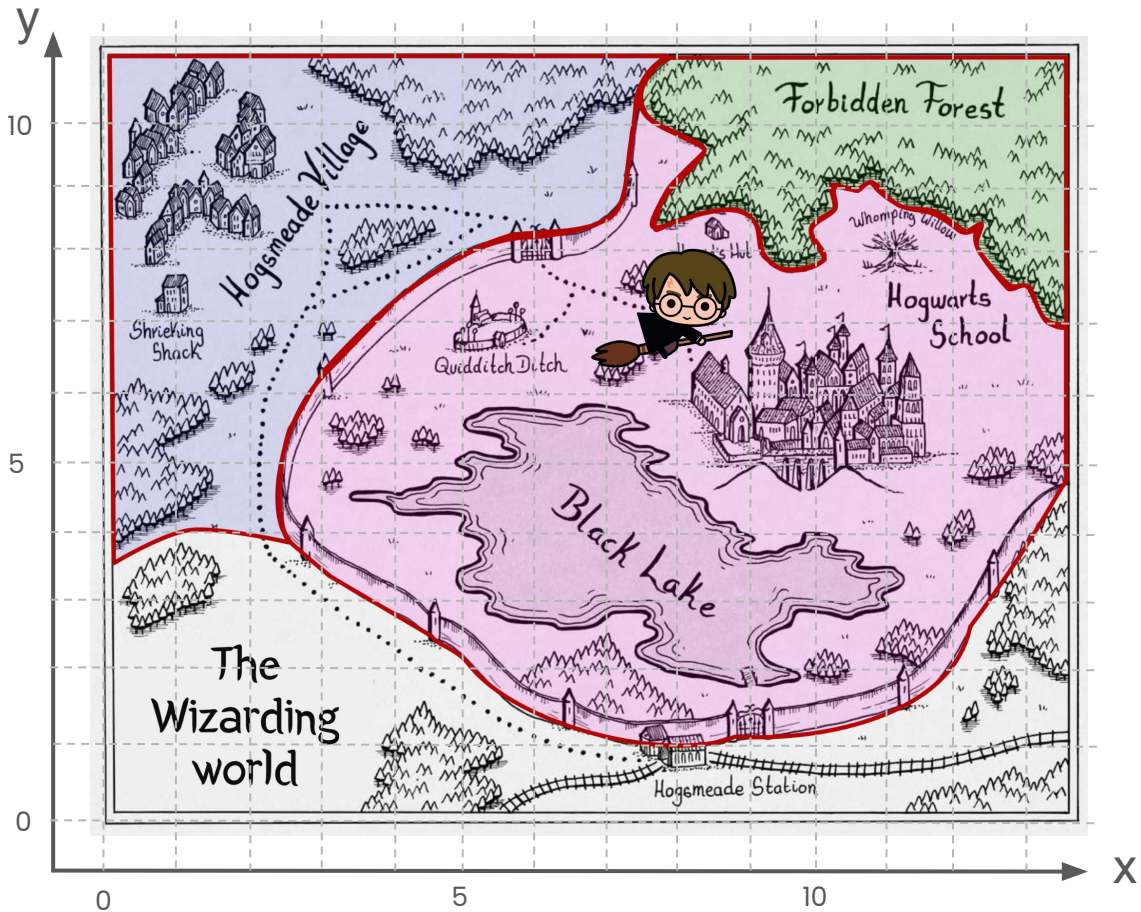
Each input corresponds to one behaviour.

Equivalence Partitioning is

grouping inputs into sets that should behave the same.

Each input belongs to one set.

Equivalence partitioning

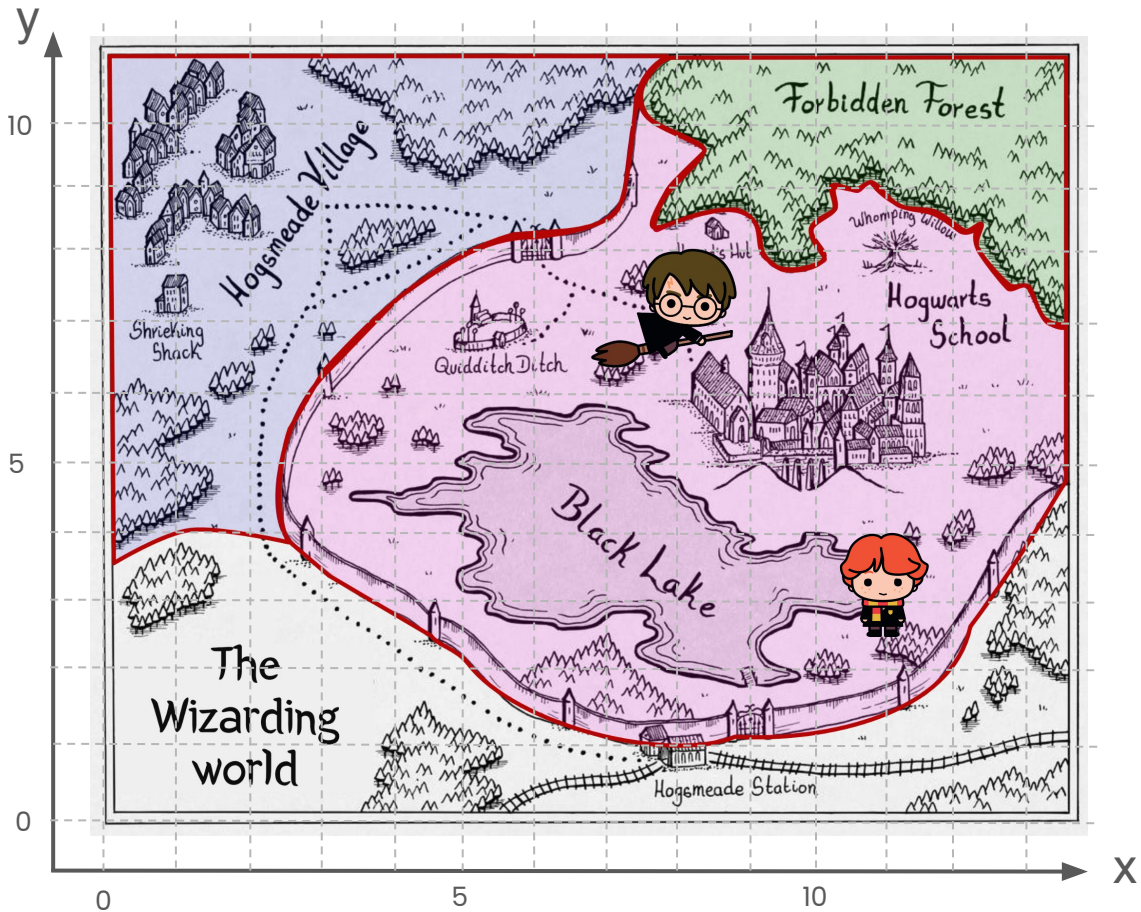


Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

Equivalence partitioning



Test cases:

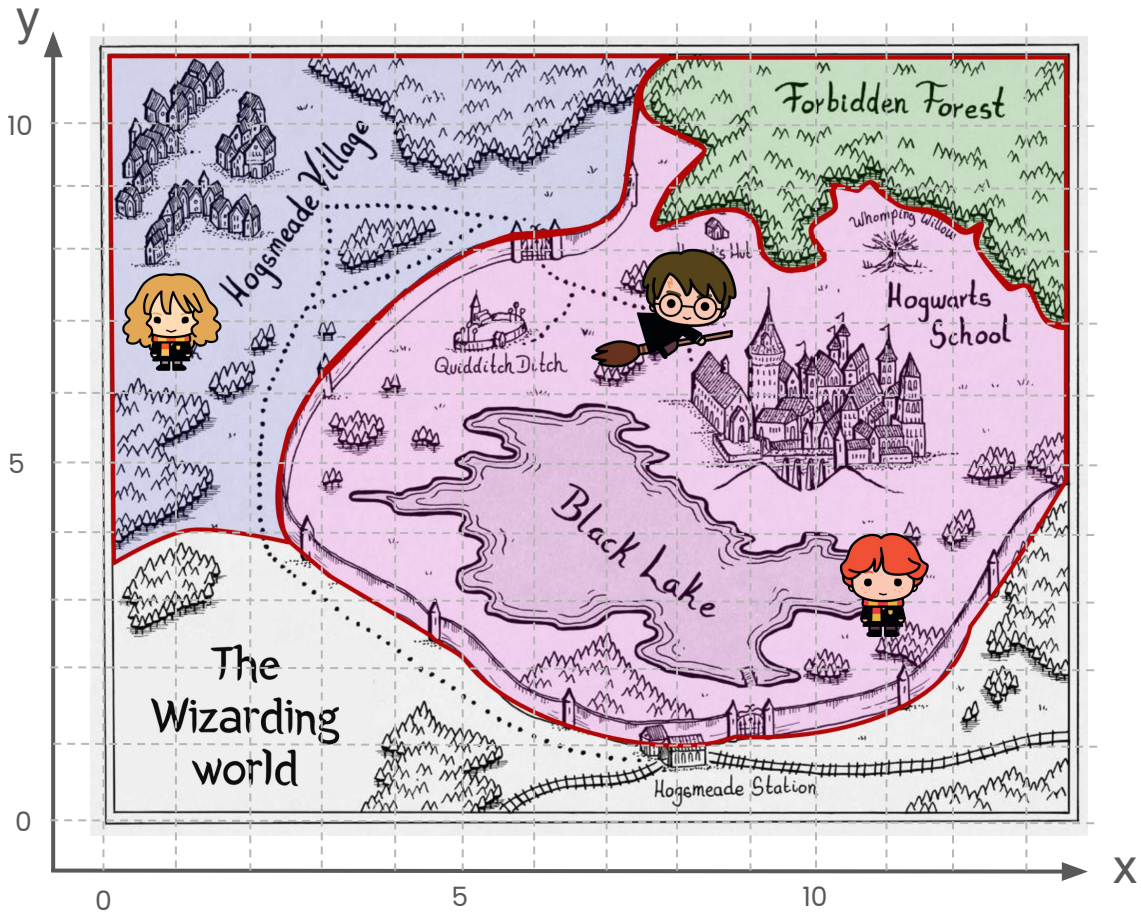
TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(11, 4)`

`=> "Hogwarts School"`

Equivalence partitioning



Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

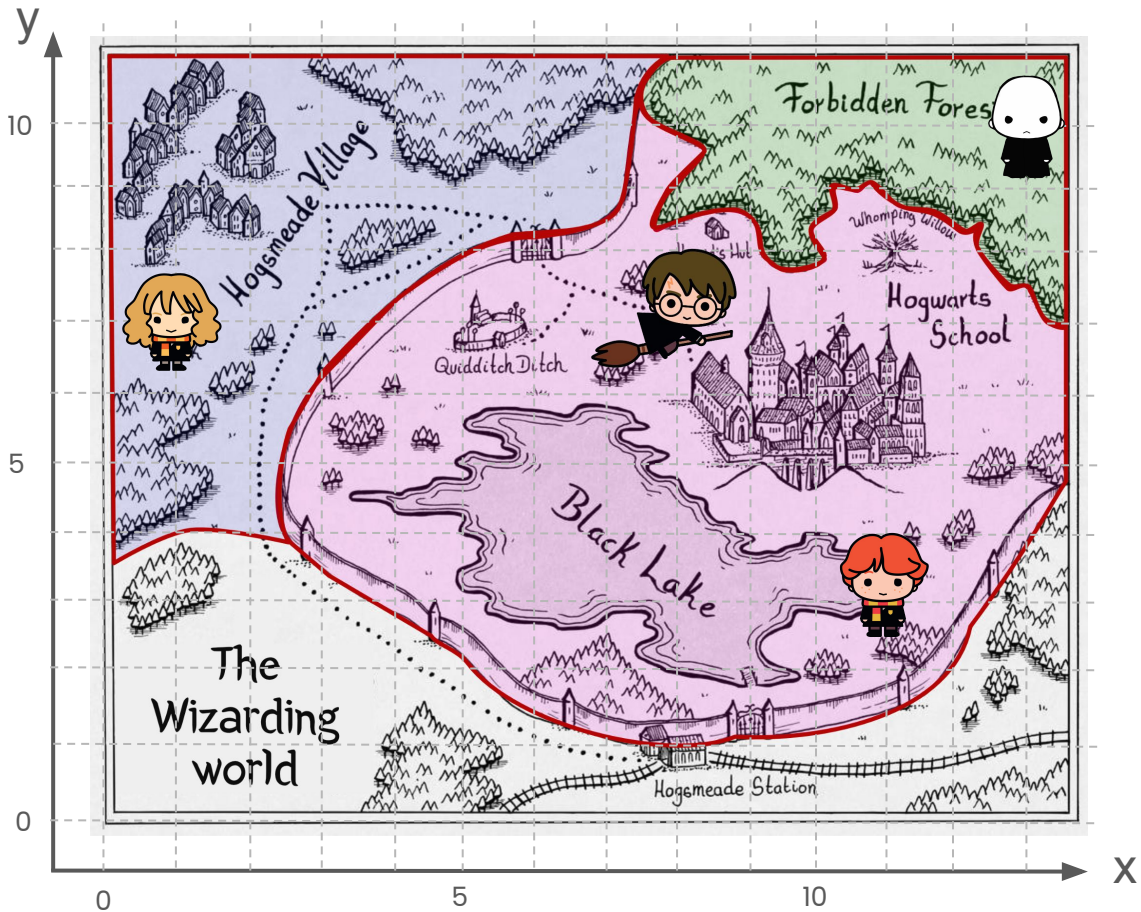
TC2: `locate_character(11, 4)`

`=> "Hogwarts School"`

TC3: `locate_character(1, 7)`

`=> "Hogsmeade Village"`

Equivalence partitioning



Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(11, 4)`

`=> "Hogwarts School"`

TC3: `locate_character(1, 7)`

`=> "Hogsmeade Village"`

TC4: `locate_character(13, 10)`

`=> "Forbidden Forest"`

What are we testing with EP?

- Whether the function behaves correctly for each region of the input domain (Expected behaviour for each partition = actual behaviour).

What are we testing with EP?

- Whether the function behaves correctly for each region of the input domain (Expected behaviour for each partition = actual behaviour).

- Whether we've covered all behaviourally different cases with as few tests as possible (Efficiency: testing sets instead of individual values).

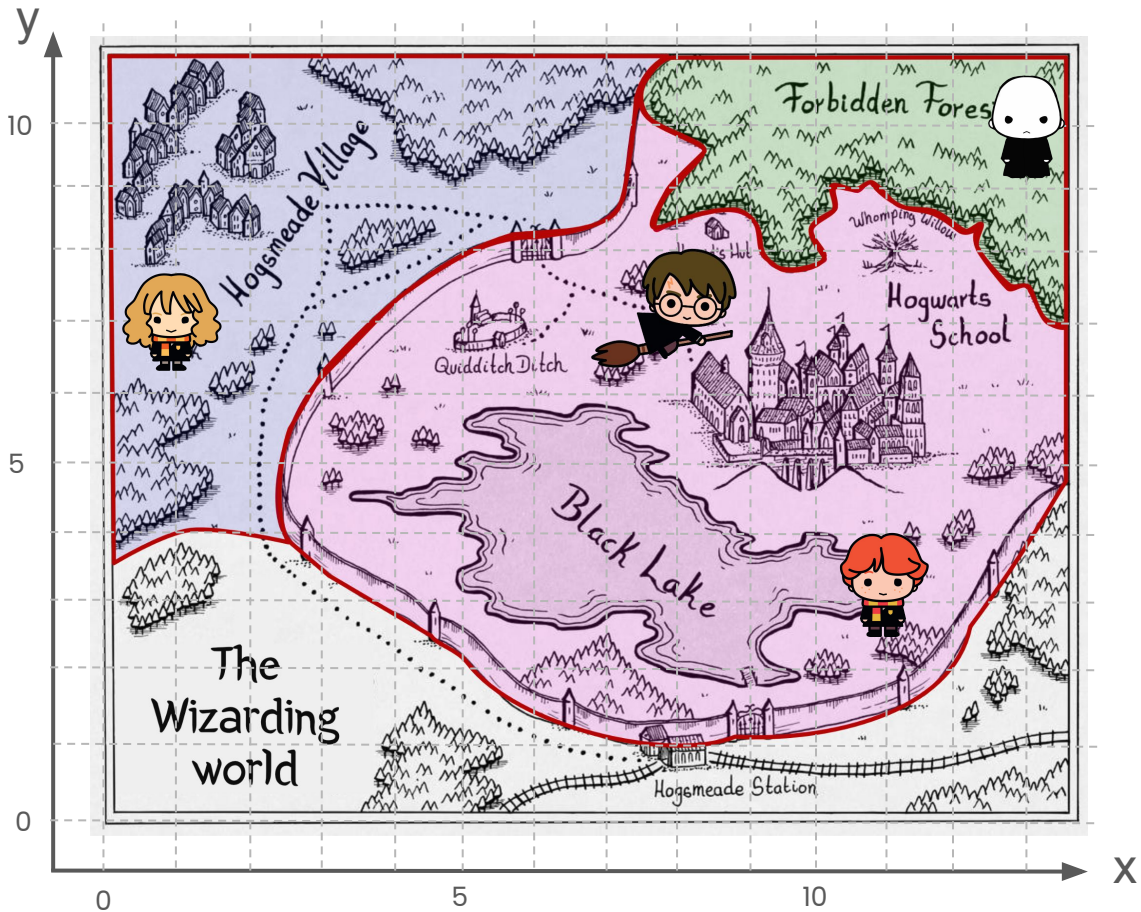
What are we testing with EP?

- Whether the function behaves correctly for each region of the input domain (Expected behaviour for each partition = actual behaviour).

- Whether we've covered all behaviourally different cases with as few tests as possible (Efficiency: testing sets instead of individual values).

- Whether the specification makes sense and is consistent (If the specification(map) has inconsistencies, EP helps reveal them).

Equivalence partitioning



Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(11, 4)`

`=> "Hogwarts School"`

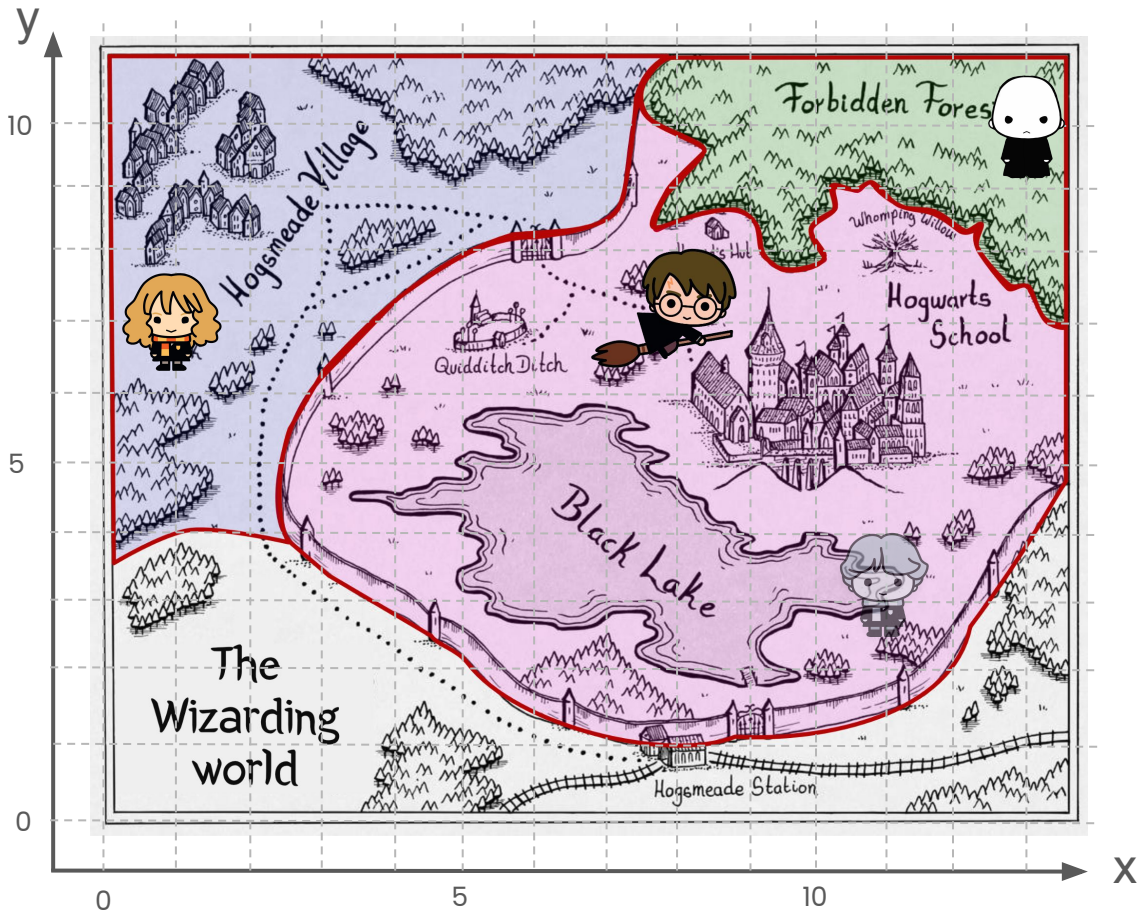
TC3: `locate_character(1, 7)`

`=> "Hogsmeade Village"`

TC4: `locate_character(13, 10)`

`=> "Forbidden Forest"`

Equivalence partitioning



Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

~~TC2: `locate_character(11, 4)`~~

~~`=> "Hogwarts School"`~~

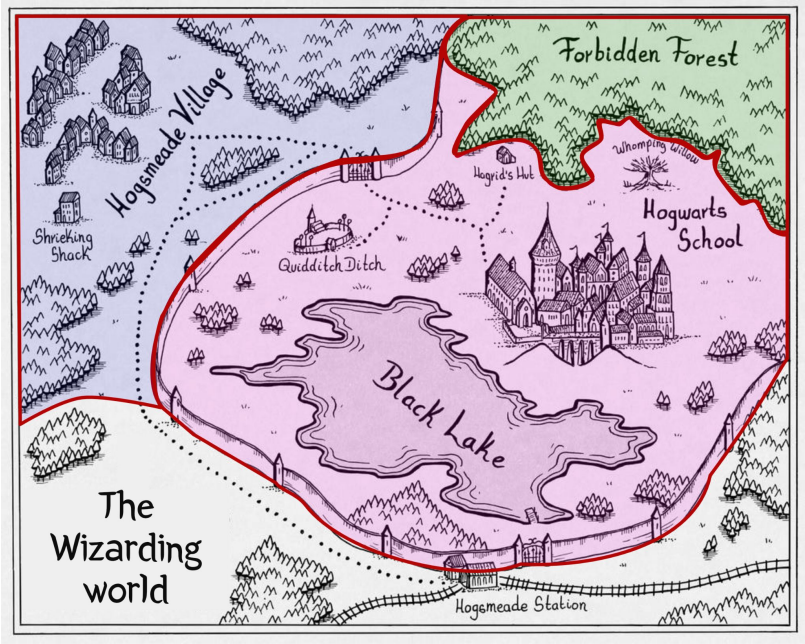
TC3: `locate_character(1, 7)`

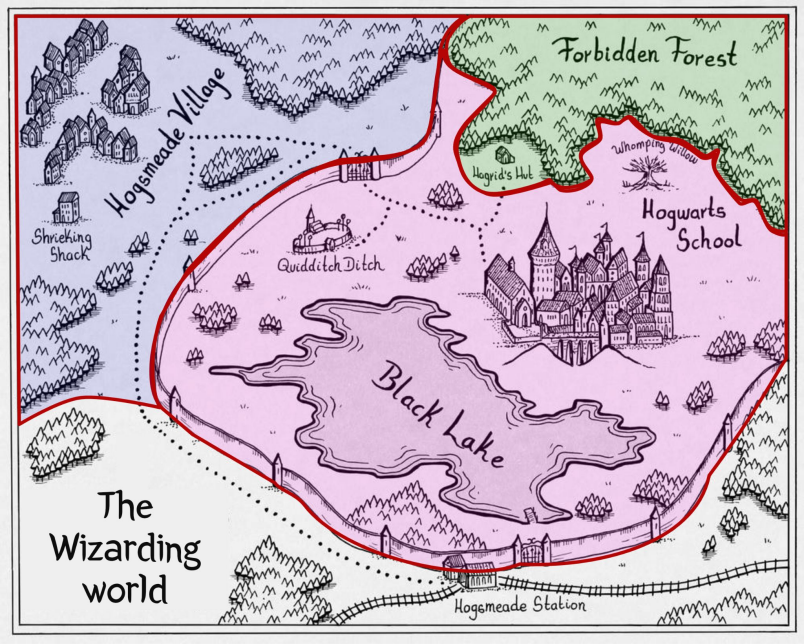
`=> "Hogsmeade Village"`

TC4: `locate_character(13, 10)`

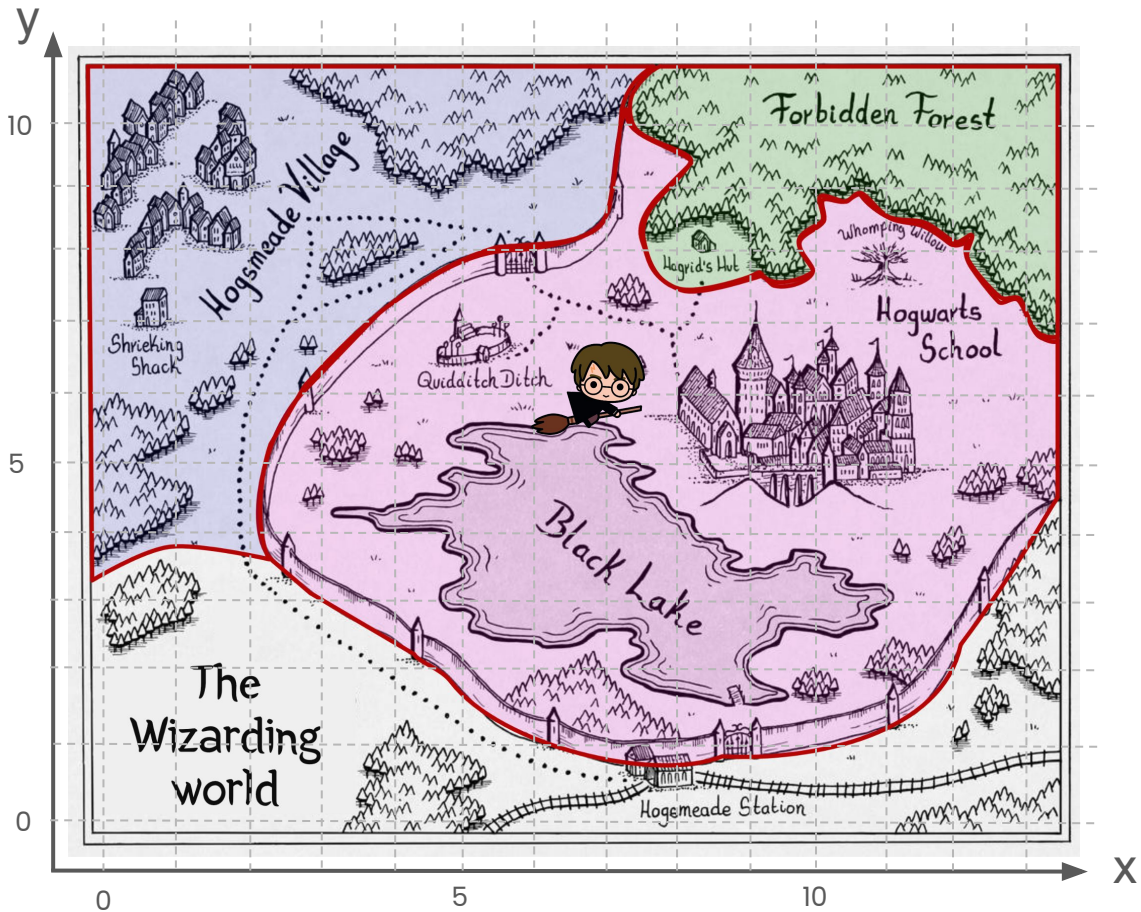
`=> "Forbidden Forest"`

Find one difference





How do we verify the borders are correct?



Test cases:

TC1: `locate_character(7, 6)`

`=> "Hogwarts School"`

How do we verify the borders are correct?

Test cases:

TC1: `locate_character(7, 6)`

`=> "Hogwarts School"`

TC2: `locate_character(8, 7)`

`=> "Hogwarts School"`

How do we verify the borders are correct?

Test cases:

TC1: `locate_character(7, 6)`

`=> "Hogwarts School"`

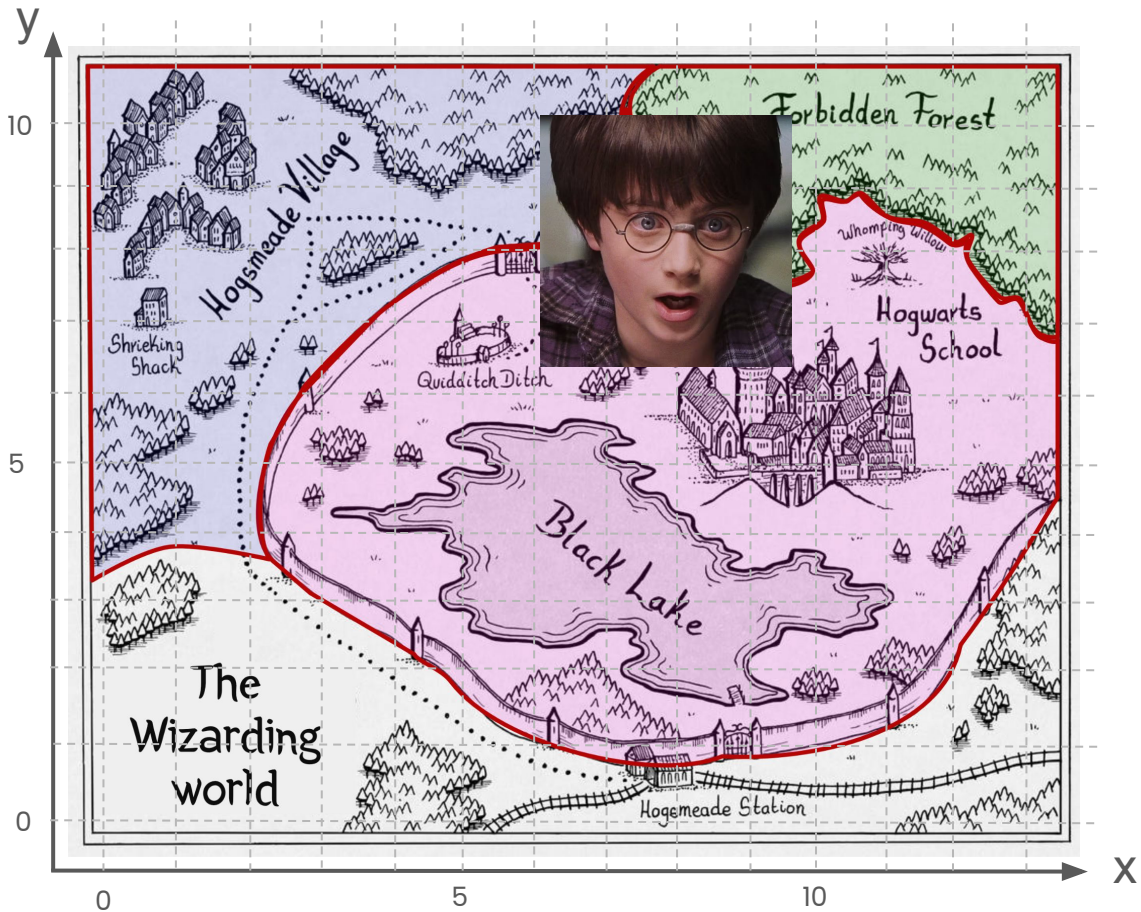
TC2: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC3: `locate_character(8, 8)`

`=> "Forbidden Forest"`

How do we verify the borders are correct?



Test cases:

TC1: `locate_character(7, 6)`

`=> "Hogwarts School"`

TC2: `locate_character(8, 7)`

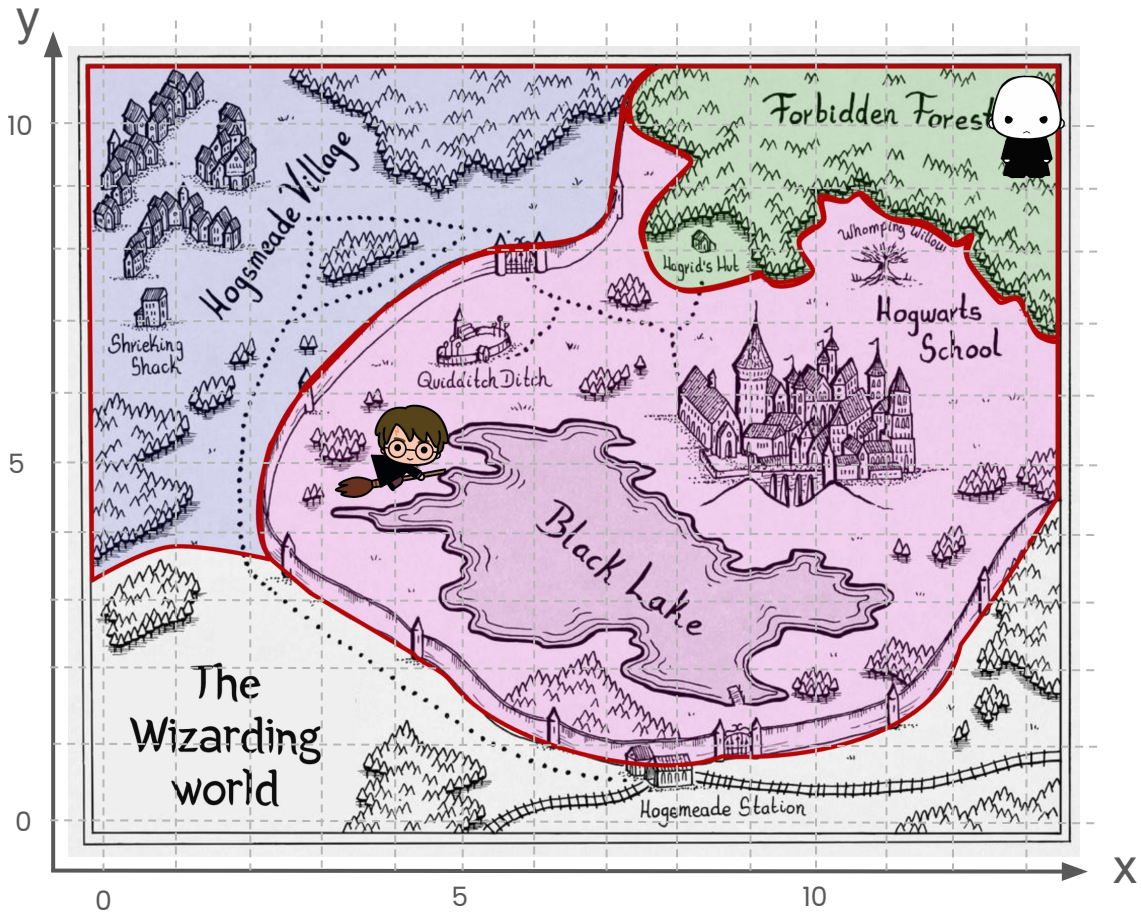
`=> "Hogwarts School"`

TC3: `locate_character(8, 8)`

`=> "Forbidden Forest"`

According to the map (specification) Hagrid's Hut is part of Hogwarts

Mission: Find the boundary between Hogwarts and Forest



Test cases:

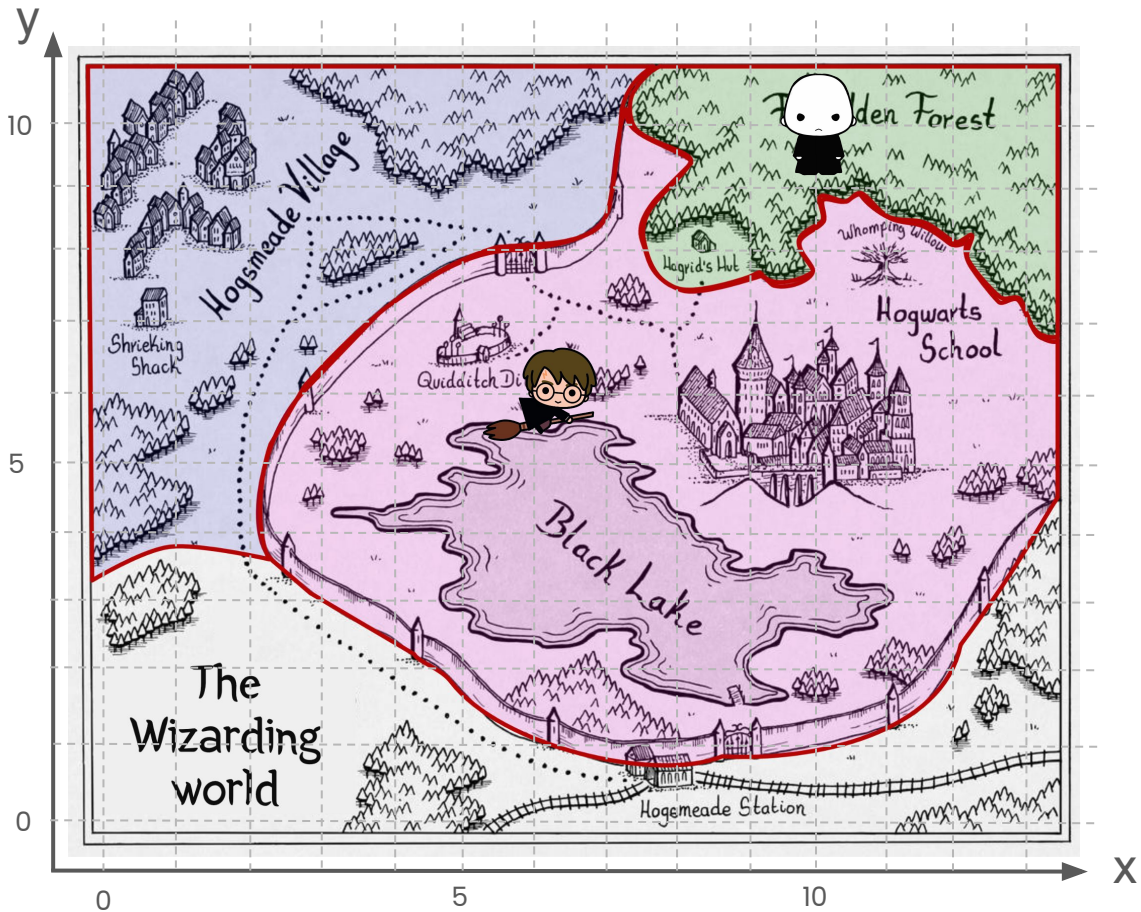
TC1: `locate_character(4, 5)`

`=> "Hogwarts School"`

TC2: `locate_character(13, 10)`

`=> "Forbidden Forest"`

Mission: Find the boundary between Hogwarts and Forest



Test cases:

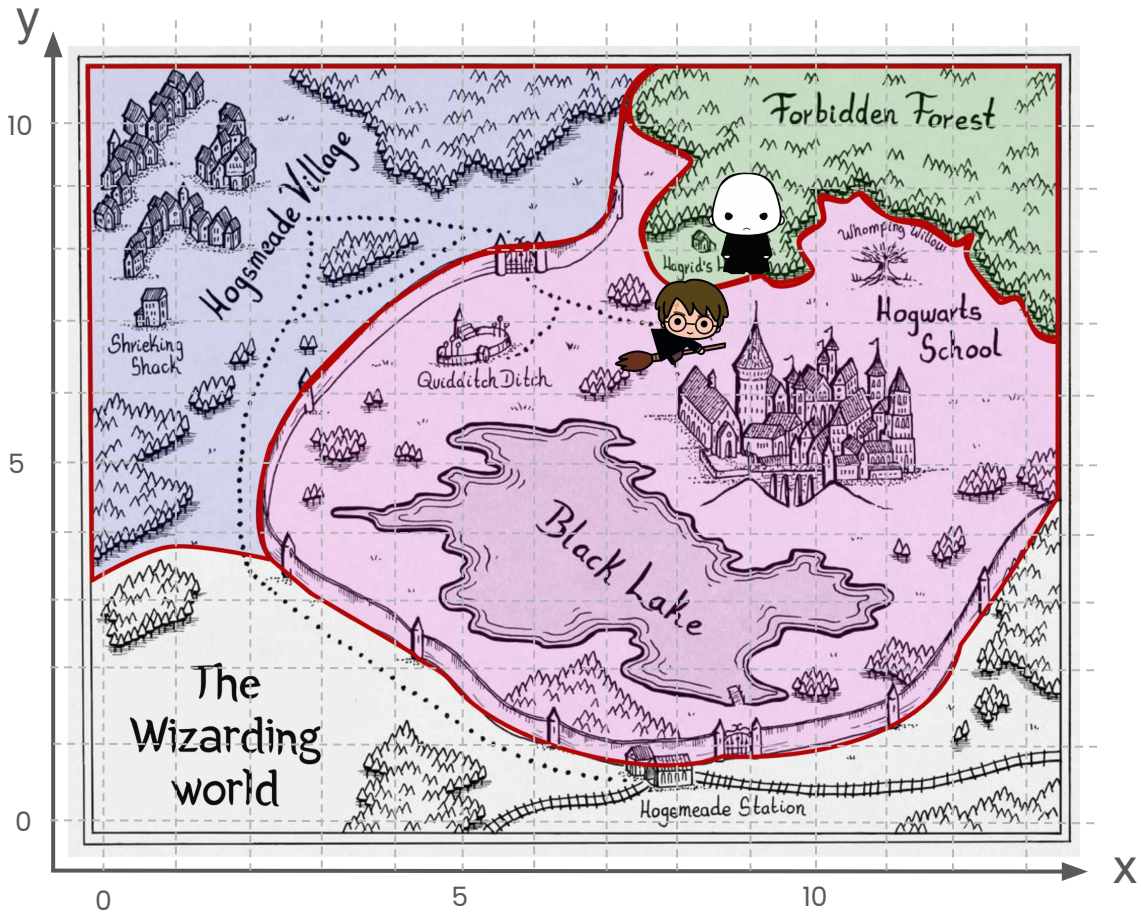
TC1: `locate_character(6, 6)`

`=> "Hogwarts School"`

TC2: `locate_character(10, 10)`

`=> "Forbidden Forest"`

Mission: Find the boundary between Hogwarts and Forest



Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(9, 8)`

`=> "Forbidden Forest"`

Mission: Find the boundary between Hogwarts and Forest

Test cases:

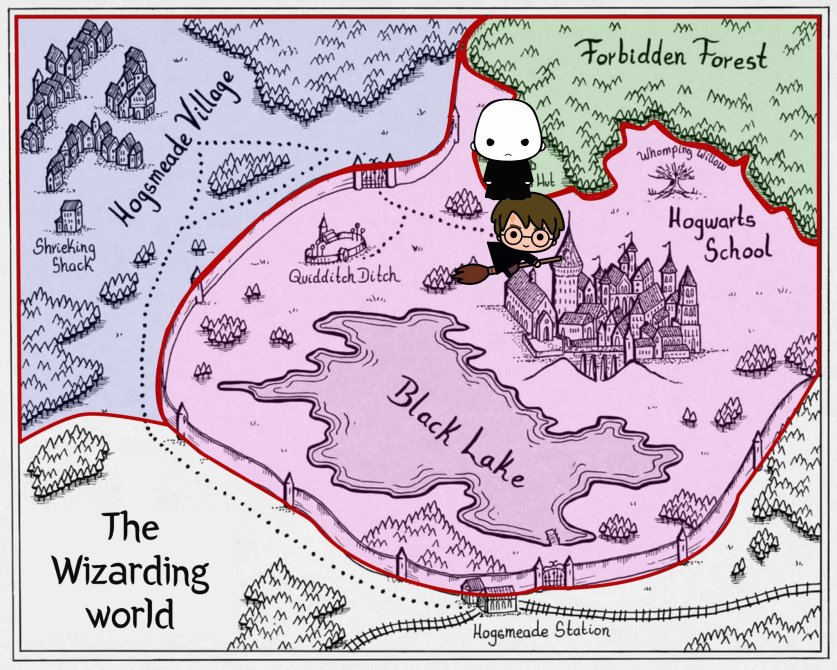
TC1: `locate_character(8, 7)`

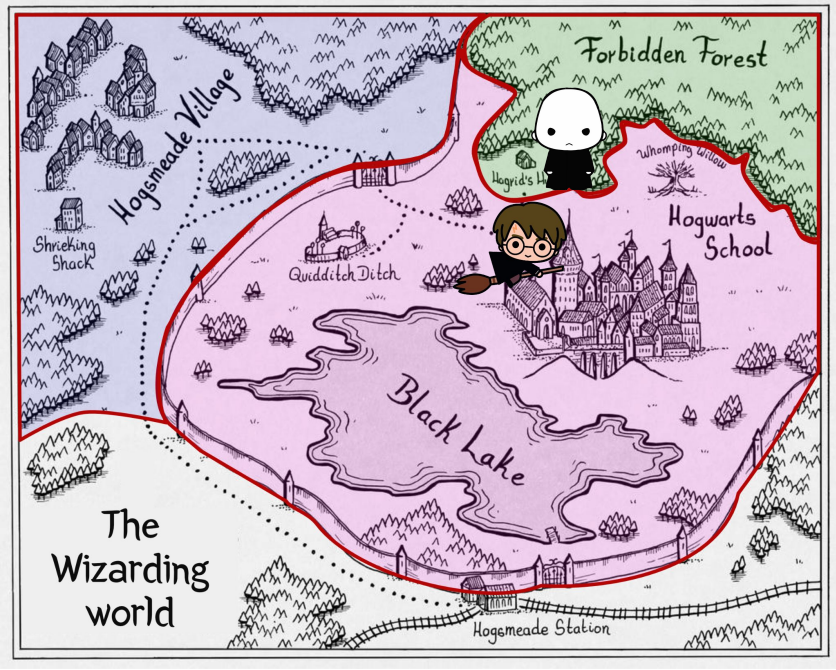
`=> "Hogwarts School"`

TC2: `locate_character(8, 8)`

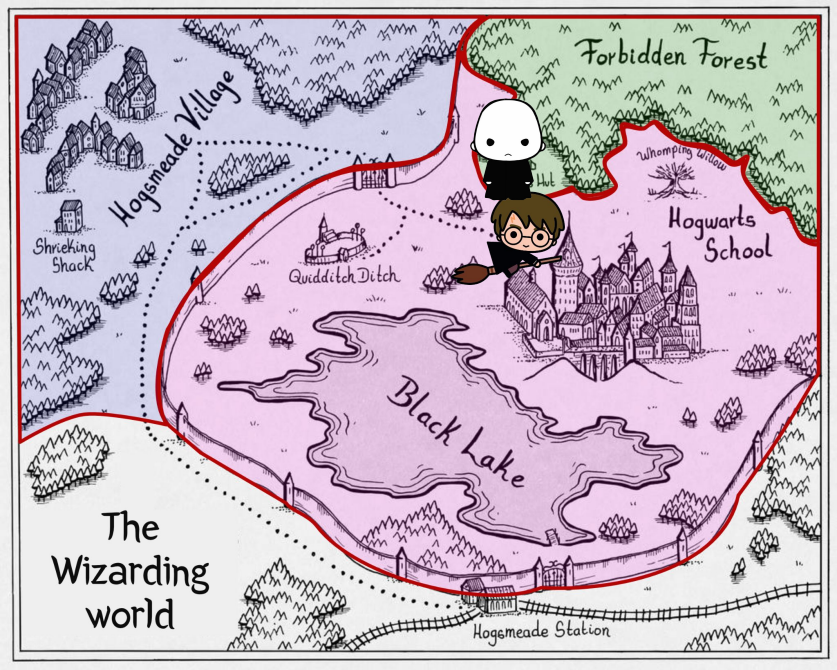
`=> "Forbidden Forest"`

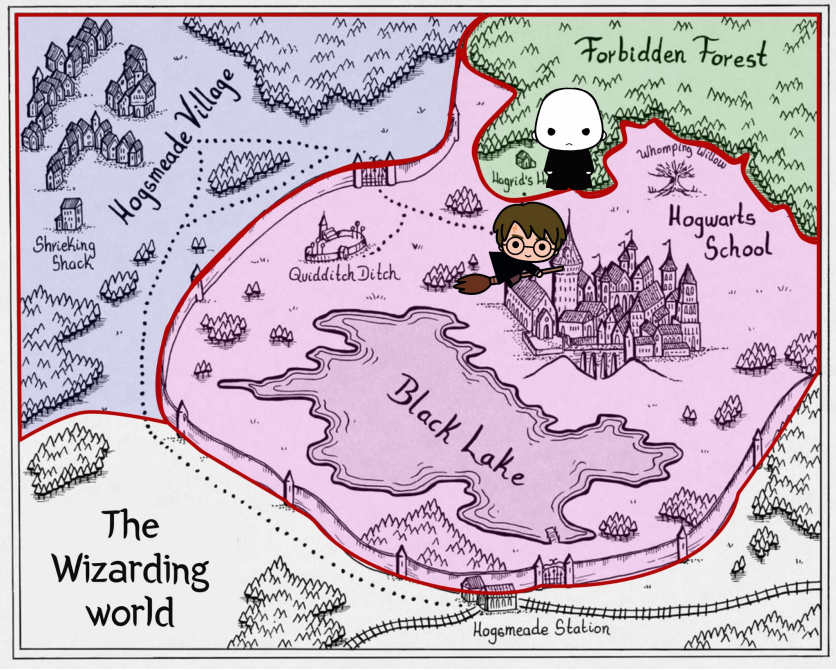
Which pair of test cases is a boundary?





Which pair of test cases is a boundary?

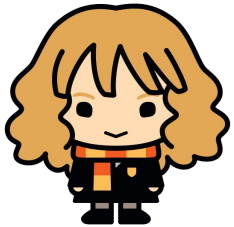




Both are boundaries, but are they equally good?

Boundary is the pair of inputs that belong to different equivalence partitions.

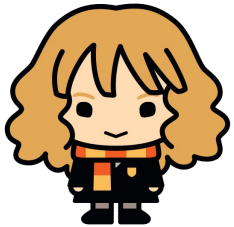
Boundary is the pair of inputs that belong to different equivalence partitions.



Boundary is always a pair!

Boundary is the pair of inputs that belong to different equivalence partitions.

We can measure how good the boundary input pair is.



Boundary is always a pair!

What is a good boundary?

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(9, 8)`

`=> "Forbidden Forest"`

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(8, 8)`

`=> "Forbidden Forest"`

What is a good boundary?

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(9, 8)`

`=> "Forbidden Forest"`

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(8, 8)`

`=> "Forbidden Forest"`

A good boundary is when inputs are as close/similar and outputs are as far/different as possible.

What is a good boundary?

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(9, 8)`

`=> "Forbidden Forest"`

Test cases:

TC1: `locate_character(8, 7)`

`=> "Hogwarts School"`

TC2: `locate_character(8, 8)`

`=> "Forbidden Forest"`

A good boundary is when inputs are as close/similar and outputs are as far/different as possible.

We care about boundary cases because mistakes usually happen where one partition changes into another, and EP alone cannot reveal those edge-behaviour errors.

We care about boundary cases because mistakes usually happen where one partition changes into another, and EP alone cannot reveal those edge-behaviour errors.

Boundaries are SUT specific. They require domain knowledge!

Boundary Value Analysis/Testing – **what**, why and how

What – BVA is a **black-box testing** technique where we choose **pairs of inputs** that trigger different behaviour.

- Each test input can be an **input vector**: a method with three input parameters, means that the boundary `[input1` and `input2]` is actually `[i1_1, i1_2, i_3]` and `[i2_1, i2_2, i2_3]`.

Boundary Value Analysis/Testing – what, why and **how**

- **What** – BVA is a **black-box testing** technique where we choose **pairs of inputs** that trigger different behaviour.
  - Each test input can be an **input vector**: a method with three input parameters, means that the boundary `[input1` and `input2]` is actually `[i1_1, i1_2, i_3]` and `[i2_1, i2_2, i2_3]`.
- **How** – After identifying partitions, we do BVT by selecting **consecutive / close inputs** that trigger behavioural change between partitions.

Boundary Value Analysis/Testing – what, **why** and how

- **What** – BVA is a **black-box testing** technique where we choose **pairs of inputs** that trigger different behaviour.
  - Each test input can be an **input vector**: a method with three input parameters, means that the boundary `[input1` and `input2]` is actually `[i1_1, i1_2, i_3]` and `[i2_1, i2_2, i2_3]`.
- **How** – After identifying partitions, we do BVT by selecting **consecutive / close inputs** that trigger behavioural change between partitions.
- **Why** – Developers often make **mistakes in conditions or control flow** that trigger behavioural changes. For instance, using `">"` instead of `">="`.

What have we learnt so far?

1. What is a boundary?
2. What are validity groups?
3. How to calculate Program Derivative?

Distance functions

- A distance function d(a, b) measures the distance between two pieces of information a and b. In our case, a and b are input and output values.
- Ideally, distances should be normalized between 0 and 1, but that is not always feasible.
  - When d(a, b) == 0, it means that a == b
  - When d(a, b) == 1, it means that a and b are completely different
- There are hundreds of distance functions, each handling a different type of information (integers, strings, vectors, images, etc.)

Exercise: Hogwarts travel rules

`can_visit(location: int, age: int) returns 0 or 1:`

1 (allowed) for:

- Hogwarts (location=0), any age
- Hogsmeade (location=1) if age ≥ 13
- Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: 1) calculate distances and 2) select TCs that test boundary.

For input distance: approximation of Euclidean distance

$$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$$

For output distance: subtraction between outputs

$$d(\text{output1}, \text{output2}) = |\text{output1} - \text{output2}|$$

Exercise: Hogwarts travel rules (distance matrix)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}= d_{out}=	-						
TC3	in:(loc=1, age=13); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	-					
TC4	in:(loc=2, age=13); out:0	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-				
TC5	in:(loc=2, age=16); out:0	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-			
TC6	in:(loc=2, age=17); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-		
TC7	in:(loc=0, age=5); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-	
TC8	in:(loc=0, age=80); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-

For input distance:

$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$

For output distance:

$d(\text{output1}, \text{output2}) = |\text{output1} - \text{output2}|$

Exercise: Hogwarts travel rules (distance matrix)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}=2 d_{out}=0	-						
TC3	in:(loc=1, age=13); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	-					
TC4	in:(loc=2, age=13); out:0	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-				
TC5	in:(loc=2, age=16); out:0	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-			
TC6	in:(loc=2, age=17); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-		
TC7	in:(loc=0, age=5); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-	
TC8	in:(loc=0, age=80); out:1	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	d_{in}= d_{out}=	-

For input distance:

$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$

For output distance:

$d(\text{output1}, \text{output2}) = |\text{output1} - \text{output2}|$

TC1 and TC2:

$d(\text{input1}, \text{input2}) = \sqrt{(1-1)^2 + (12-10)^2} = 2$

$d(\text{output1}, \text{output2}) = |0 - 0| = 0$

Exercise: Hogwarts travel rules (distance matrix)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}=2 d_{out}=0	-						
TC3	in:(loc=1, age=13); out:1	d_{in}=3 d_{out}=1	d_{in}=1 d_{out}=1	-					
TC4	in:(loc=2, age=13); out:0	d_{in}=3 d_{out}=0	d_{in}=1 d_{out}=0	d_{in}=1 d_{out}=1	-				
TC5	in:(loc=2, age=16); out:0	d_{in}=6 d_{out}=0	d_{in}=4 d_{out}=0	d_{in}=3 d_{out}=1	d_{in}=3 d_{out}=0	-			
TC6	in:(loc=2, age=17); out:1	d_{in}=7 d_{out}=1	d_{in}=5 d_{out}=1	d_{in}=4 d_{out}=0	d_{in}=4 d_{out}=1	d_{in}=1 d_{out}=1	-		
TC7	in:(loc=0, age=5); out:1	d_{in}=5 d_{out}=1	d_{in}=7 d_{out}=1	d_{in}=8 d_{out}=0	d_{in}=8 d_{out}=1	d_{in}= 11 d_{out}=1	d_{in}=12 d_{out}=0	-	
TC8	in:(loc=0, age=80); out:1	d_{in}=70 d_{out}=1	d_{in}=68 d_{out}=1	d_{in}=67 d_{out}=0	d_{in}=67 d_{out}=1	d_{in}=64 d_{out}=1	d_{in}=63 d_{out}=0	d_{in}=75 d_{out}=0	-

can_visit(location: int, age: int) returns 0 or 1

1 (allowed) for:

Hogwarts (location=0), any age

Hogsmeade (location=1) if age ≥ 13

Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: select TCs that test boundary.

Exercise: Hogwarts travel rules (boundary candidates)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}=2 d_{out}=0	-						
TC3	in:(loc=1, age=13); out:1	d_{in}=3 d_{out}=1	d_{in}=1 d_{out}=1	-					
TC4	in:(loc=2, age=13); out:0	d_{in}=3 d_{out}=0	d_{in}=1 d_{out}=0	d_{in}=1 d_{out}=1	-				
TC5	in:(loc=2, age=16); out:0	d_{in}=6 d_{out}=0	d_{in}=4 d_{out}=0	d_{in}=3 d_{out}=1	d_{in}=3 d_{out}=0	-			
TC6	in:(loc=2, age=17); out:1	d_{in}=7 d_{out}=1	d_{in}=5 d_{out}=1	d_{in}=4 d_{out}=0	d_{in}=4 d_{out}=1	d_{in}=1 d_{out}=1	-		
TC7	in:(loc=0, age=5); out:1	d_{in}=5 d_{out}=1	d_{in}=7 d_{out}=1	d_{in}=8 d_{out}=0	d_{in}=8 d_{out}=1	d_{in}= 11 d_{out}=1	d_{in}=12 d_{out}=0	-	
TC8	in:(loc=0, age=80); out:1	d_{in}=70 d_{out}=1	d_{in}=68 d_{out}=1	d_{in}=67 d_{out}=0	d_{in}=67 d_{out}=1	d_{in}=64 d_{out}=1	d_{in}=63 d_{out}=0	d_{in}=75 d_{out}=0	-

can_visit(location: int, age: int) returns 0 or 1

1 (allowed) for:

Hogwarts (location=0), any age

Hogsmeade (location=1) if age ≥ 13

Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Exercise: Hogwarts travel rules (boundaries)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}=2 d_{out}=0	-						
TC3	in:(loc=1, age=13); out:1	d_{in}=3 d_{out}=1	d_{in}=1 d_{out}=1	-					
TC4	in:(loc=2, age=13); out:0	d_{in}=3 d_{out}=0	d_{in}=1 d_{out}=0	d_{in}=1 d_{out}=1	-				
TC5	in:(loc=2, age=16); out:0	d_{in}=6 d_{out}=0	d_{in}=4 d_{out}=0	d_{in}=3 d_{out}=1	d_{in}=3 d_{out}=0	-			
TC6	in:(loc=2, age=17); out:1	d_{in}=7 d_{out}=1	d_{in}=5 d_{out}=1	d_{in}=4 d_{out}=0	d_{in}=4 d_{out}=1	d_{in}=1 d_{out}=1	-		
TC7	in:(loc=0, age=5); out:1	d_{in}=5 d_{out}=1	d_{in}=7 d_{out}=1	d_{in}=8 d_{out}=0	d_{in}=8 d_{out}=1	d_{in}= 11 d_{out}=1	d_{in}=12 d_{out}=0	-	
TC8	in:(loc=0, age=80); out:1	d_{in}=70 d_{out}=1	d_{in}=68 d_{out}=1	d_{in}=67 d_{out}=0	d_{in}=67 d_{out}=1	d_{in}=64 d_{out}=1	d_{in}=63 d_{out}=0	d_{in}=75 d_{out}=0	-

can_visit(location: int, age: int) returns 0 or 1

1 (allowed) for:

Hogwarts (location=0), any age

Hogsmeade (location=1) if age ≥ 13

Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Exercise: Hogwarts travel rules (boundaries)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	d_{in}=2 d_{out}=0	-						
TC3	in:(loc=1, age=13); out:1	d_{in}=3 d_{out}=1	d_{in}=1 d_{out}=1	-					
TC4	in:(loc=2, age=13); out:0	d_{in}=3 d_{out}=0	d_{in}=1 d_{out}=0	d_{in}=1 d_{out}=1	-				
TC5	in:(loc=2, age=16); out:0	d_{in}=6 d_{out}=0	d_{in}=4 d_{out}=0	d_{in}=3 d_{out}=1	d_{in}=3 d_{out}=0				
TC6	in:(loc=2, age=17); out:1	d_{in}=7 d_{out}=1	d_{in}=5 d_{out}=1	d_{in}=4 d_{out}=0	d_{in}=4 d_{out}=1	d_{in}=1 d_{out}=1	-		
TC7	in:(loc=0, age=5); out:1	d_{in}=5 d_{out}=1	d_{in}=7 d_{out}=1	d_{in}=8 d_{out}=0	d_{in}=8 d_{out}=1	d_{in}= 11 d_{out}=1	d_{in}=12 d_{out}=0	-	
TC8	in:(loc=0, age=80); out:1	d_{in}=70 d_{out}=1	d_{in}=68 d_{out}=1	d_{in}=67 d_{out}=0	d_{in}=67 d_{out}=1	d_{in}=64 d_{out}=1	d_{in}=63 d_{out}=0	d_{in}=75 d_{out}=0	-

TC2 – in:(loc=1, age=12); out:0
TC3 – in:(loc=1, age=13); out:1

TC3 – in:(loc=1, age=13); out:1
TC4 – in:(loc=2, age=13); out:0

TC5 – in:(loc=2, age=16); out:0
TC6 – in:(loc=2, age=17); out:1

Exercise: Hogwarts travel rules

```
can_visit(location: int, age: int) returns 0 or 1:
```

1 (allowed) for:

- Hogwarts (location=0), any age
- Hogsmeade (location=1) if age ≥ 13
- Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: Select TCs that test boundary.

Boundary 1:

TC2 – in:(loc=1, age=12); out:0
TC3 – in:(loc=1, age=13); out:1

Boundary 2:

TC3 – in:(loc=1, age=13); out:1
TC4 – in:(loc=2, age=13); out:0

Boundary 3:

TC5 – in:(loc=2, age=16); out:0
TC6 – in:(loc=2, age=17); out:1

For input distance: approximation of Euclidean distance

$$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$$

For output distance: subtraction between outputs

$$d(\text{output1}, \text{outpu2}) = |\text{output1} - \text{output2}|$$

Exercise: Hogwarts travel rules

```
can_visit(location: int, age: int) returns 0 or 1:
```

1 (allowed) for:

- Hogwarts (location=0), any age
- Hogsmeade (location=1) if age ≥ 13
- Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: Select TCs that test boundary.

Boundary 1:

TC2 – in:(loc=1, age=12); out:0
TC3 – in:(loc=1, age=13); out:1

TC3 – in:(loc=1, age=13); out:1
TC4 – in:(loc=2, age=13); out:0

Boundary 3:

TC5 – in:(loc=2, age=16); out:0
TC6 – in:(loc=2, age=17); out:1

For input distance: approximation of Euclidean distance

$$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$$

For output distance: subtraction between outputs

$$d(\text{output1}, \text{outpu2}) = |\text{output1} - \text{output2}|$$

Exercise: Hogwarts travel rules

```
can_visit(location: int, age: int) returns 0 or 1:
```

1 (allowed) for:

- Hogwarts (location=0), any age
- Hogsmeade (location=1) if age ≥ 13
- Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: Select TCs that test boundary.

TC2 – in:(loc=1, age=12); out:0
TC3 – in:(loc=1, age=13); out:1

TC3 – in:(loc=1, age=13); out:1
TC4 – in:(loc=2, age=13); out:0

TC5 – in:(loc=2, age=16); out:0
TC6 – in:(loc=2, age=17); out:1

For input distance: approximation of Euclidean distance

$$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$$

For output distance: subtraction between outputs

$$d(\text{output1}, \text{outpu2}) = |\text{output1} - \text{output2}|$$

Exercise: Hogwarts travel rules

```
can_visit(location: int, age: int) returns 0 or 1:
```

1 (allowed) for:

- Hogwarts (location=0), any age
- Hogsmeade (location=1) if age ≥ 13
- Forbidden Forest (location=2) if age ≥ 17

0 (not allowed) otherwise

Task: Select TCs that test boundary.

TC2 – in:(loc=1, age=12); out:0
TC3 – in:(loc=1, age=13); out:1

TC3 – in:(loc=1, age=13); out:1
TC4 – in:(loc=2, age=13); out:0

TC5 – in:(loc=2, age=16); out:0
TC6 – in:(loc=2, age=17); out:1

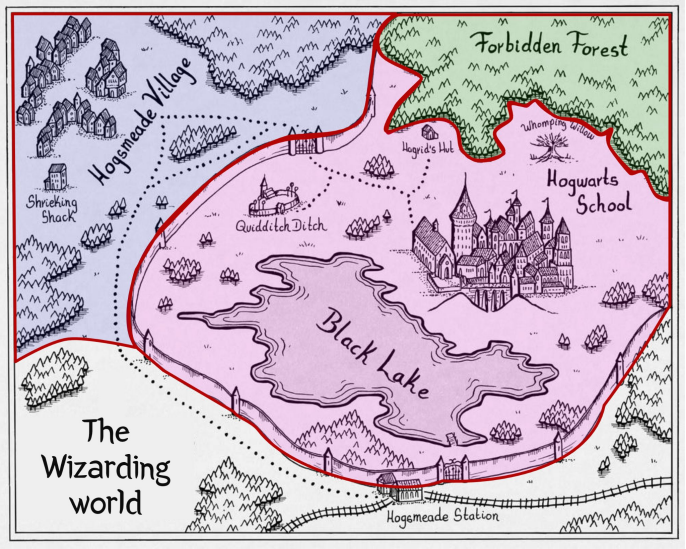
For input distance: approximation of Euclidean distance

$$d(\text{input1}, \text{input2}) = \sqrt{(\text{location1} - \text{location2})^2 + (\text{age1}-\text{age2})^2}$$

For output distance: subtraction between outputs

$$d(\text{output1}, \text{outpu2}) = |\text{output1} - \text{output2}|$$

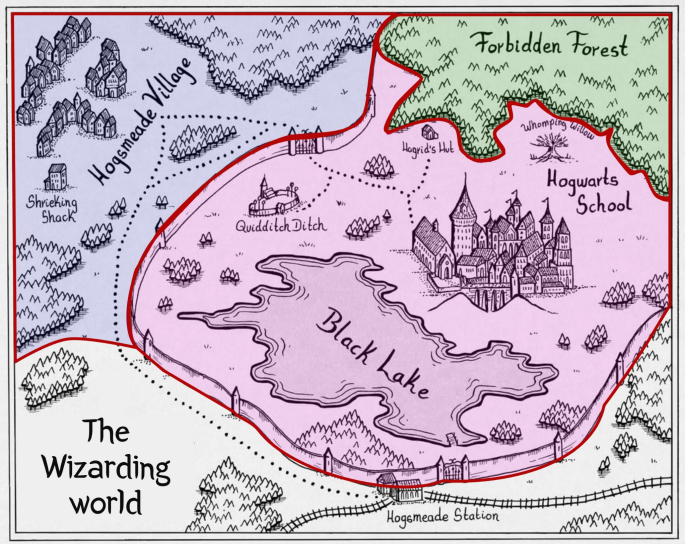
Validity groups = types of boundaries



Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Validity groups = types of boundaries



Boundaries are between partitions:

Village (valid) and Hogwarts (valid)

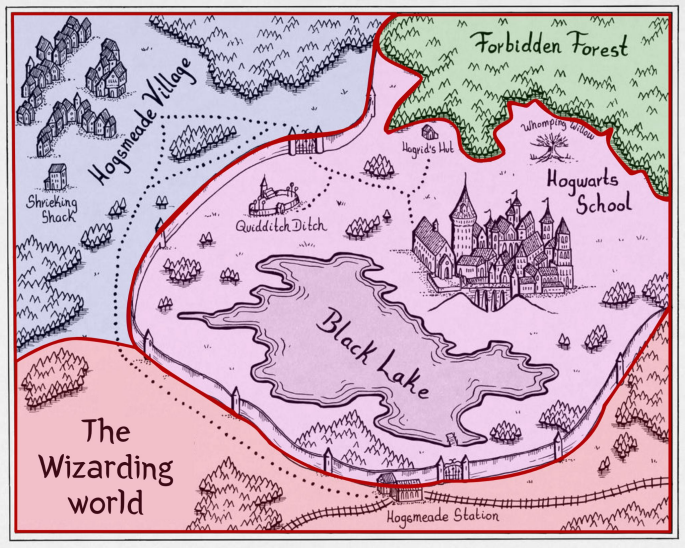
Village (valid) and Forest (valid)

Forest (valid) and Hogwarts (valid)

Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Validity groups = types of boundaries



Boundaries are between partitions:

Village (valid) and Hogwarts (valid)

Village (valid) and Forest (valid)

Forest (valid) and Hogwarts (valid)

Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Error output: "Red zone".

Validity groups = types of boundaries

Boundaries are between partitions:

Village (valid) and Hogwarts (valid)

Village (valid) and Forest (valid)

Forest (valid) and Hogwarts (valid)

Hogwarts (valid) and Red zone (error)

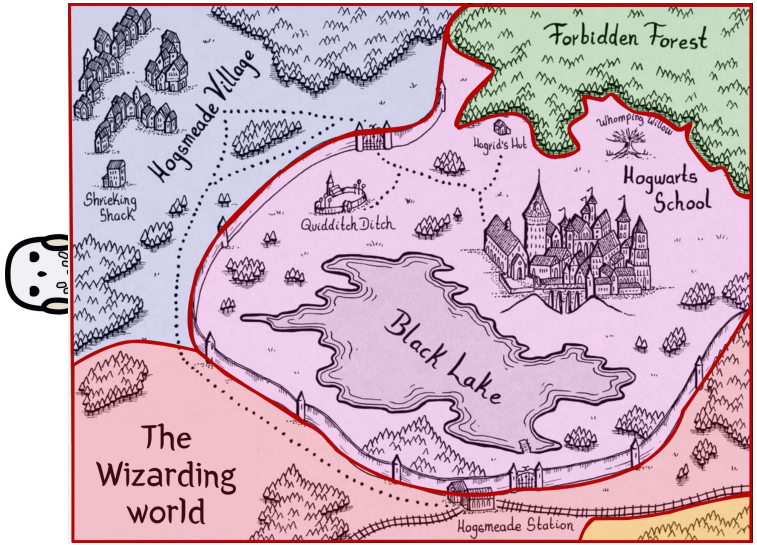
Village (valid) and Red zone (error)

Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Error output: "Red zone".

Validity groups = types of boundaries



Boundaries are between partitions:

Village (valid) and Hogwarts (valid)

Village (valid) and Forest (valid)

Forest (valid) and Hogwarts (valid)

Hogwarts (valid) and Red zone (error)

Village (valid) and Red zone (error)

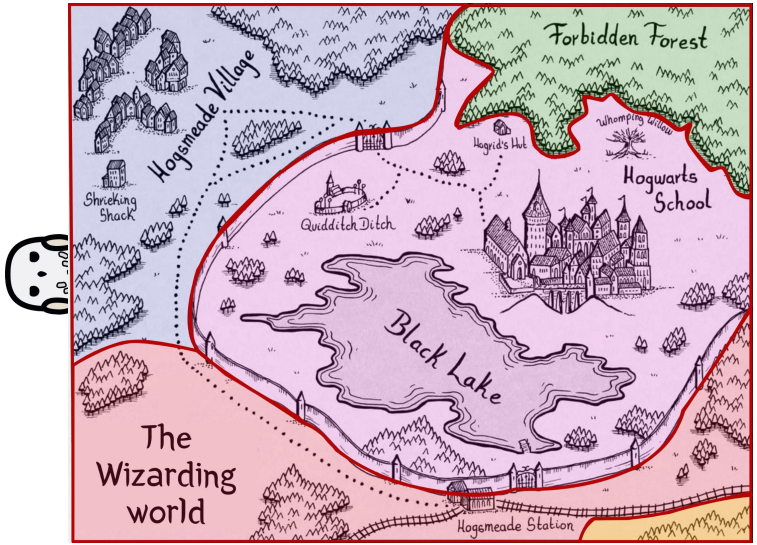
Red zone (error) and Orange zone (error)

Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Error outputs: "Red zone", "Orange zone"

Validity groups = types of boundaries



Boundaries are between partitions:

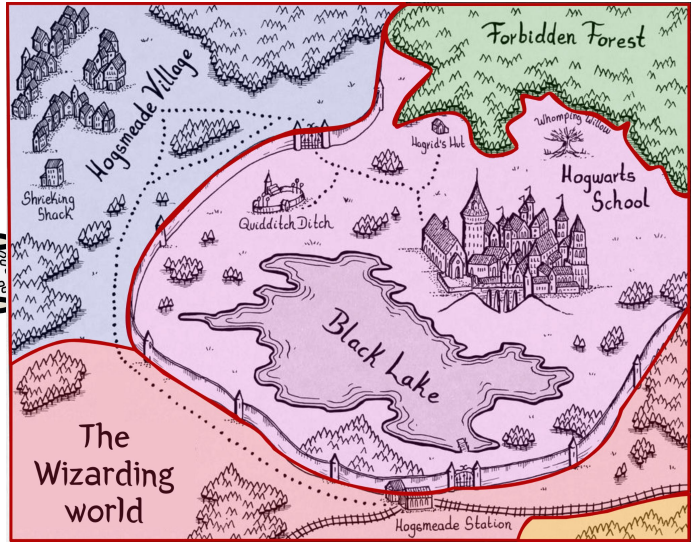
- VV
  - Village (valid) and Hogwarts (valid)
  - Village (valid) and Forest (valid)
  - Forest (valid) and Hogwarts (valid)
- VE
  - Hogwarts (valid) and Red zone (error)
  - Village (valid) and Red zone (error)
- EE
  - Red zone (error) and Orange zone (error)

Valid outputs for `locate_character`:

"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

Error outputs: "Red zone", "Orange zone"

Validity groups = types of boundaries



Boundaries are between partitions:

VV
- Village (valid) and Hogwarts (valid)
- Village (valid) and Forest (valid)
- Forest (valid) and Hogwarts (valid)

VE
- Hogwarts (valid) and Red zone (error)
- Village (valid) and Red zone (error)

EE
- Red zone (error) and Orange zone (error)

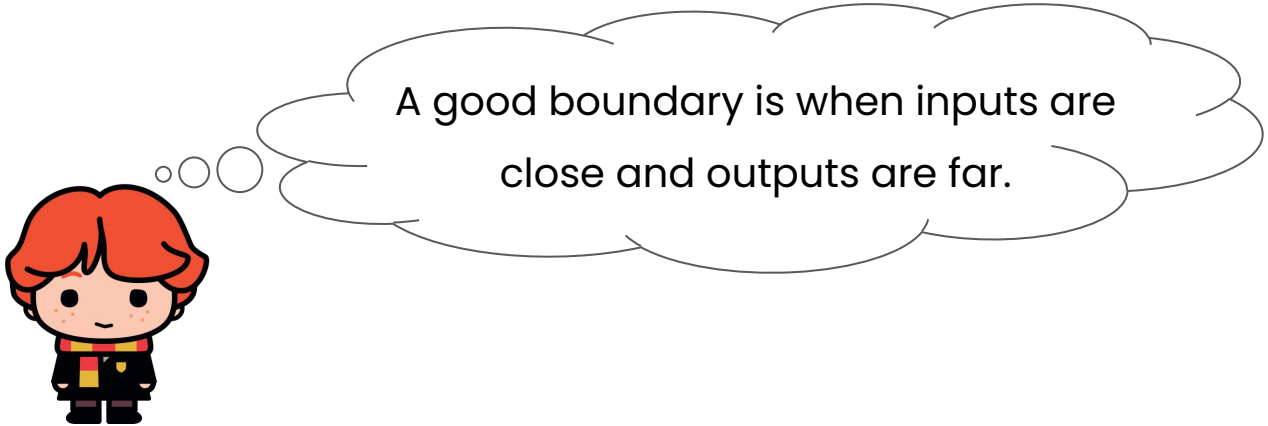
Valid outputs for `locate_character`:

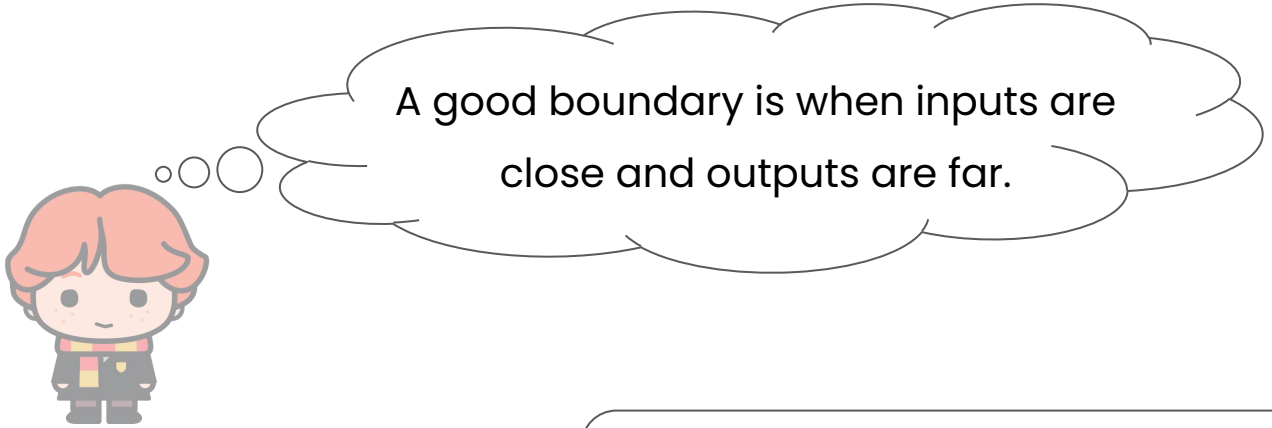
"Hogsmeade Village", "Hogwarts School", "Forbidden Forest".

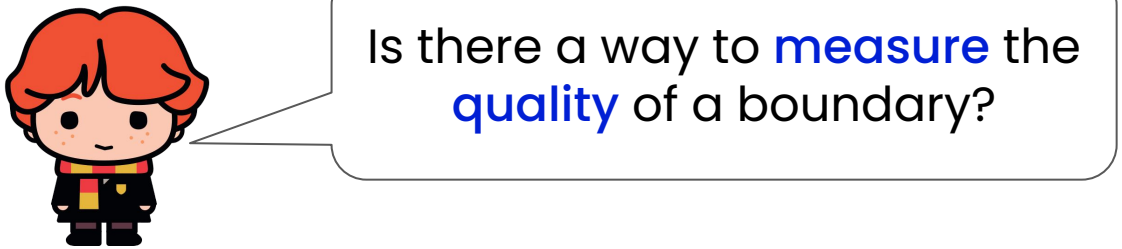
Error outputs: "Red zone", "Orange zone"

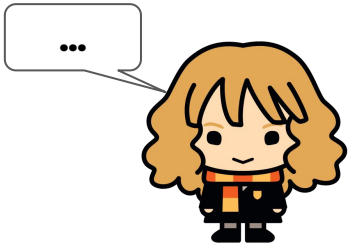
Validity groups are high-level boundary categories.

There are three validity groups: VV, VE, EE.


A good boundary is when inputs are close and outputs are far.


A good boundary is when inputs are close and outputs are far.

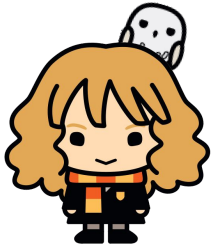

Is there a way to measure the quality of a boundary?


...

A good boundary pair is when inputs are close and outputs are far.

Is there a way to **measure** the **quality** of a boundary?

Yes, there is a metric called **Program Derivative.**



Program Derivative (PD) is a measure of

boundariness of an input pair.

In other worlds, PD **measures the quality** of a

boundary candidate.

Program Derivative (PD)

- Main idea: Boundary values can be found by **identifying similar inputs that trigger very different outputs**.
- In calculus, a derivative is a value that indicates **sensitivity to change** in a given function.

Program Derivative (PD)

- Main idea: Boundary values can be found by **identifying similar inputs that trigger very different outputs**.
- In calculus, a derivative is a value that indicates **sensitivity to change** in a given function.

$$\boxed{ProgramDerivative(input1, input2) = \frac{distance(output1, output2)}{distance(input1, input2)}}$$

Program Derivative (PD)

- Main idea: Boundary values can be found by **identifying similar inputs that trigger very different outputs**.
- In calculus, a derivative is a value that indicates **sensitivity to change** in a given function.

$$\boxed{ProgramDerivative(input1, input2) = \frac{distance(output1, output2)}{distance(input1, input2)}}$$

- Important: ***input* is a vector** of the parameters (can be multiple values).

Program Derivative (PD)

- Main idea: Boundary values can be found by **identifying similar inputs that trigger very different outputs**.
- In calculus, a derivative is a value that indicates **sensitivity to change** in a given function.

$$\boxed{ProgramDerivative(input1, input2) = \frac{distance(output1, output2)}{distance(input1, input2)}}$$

- Important: ***input* is a vector** of the parameters (can be multiple values).
- PD can take **different range of values** depending on the distance functions.

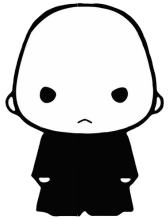
Exercise: Hogwarts travel rules (PD matrix)

		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
TC1	in:(loc=1, age=10); out:0	-							
TC2	in:(loc=1, age=12); out:0	0	-						
TC3	in:(loc=1, age=13); out:1	0.33	1	-					
TC4	in:(loc=2, age=13); out:0	0	0	1	-				
TC5	in:(loc=2, age=16); out:0	0	0	0.33	0	-			
TC6	in:(loc=2, age=17); out:1	0.14	0.2	0	0.25	1	-		
TC7	in:(loc=0, age=5); out:1	0.2	0.14	0	0.125	0.09	0	-	
TC8	in:(loc=0, age=80); out:1	0.014	0.015	0	0.015	0.016	0	0	-

Higher PD means better boundariness!

What have we learnt so far?

1. What is a boundary?
2. What are validity groups?
3. How to calculate Program Derivative?



So what? Why care about PD?

PD was introduced in 2019 (quite recently!), which enabled automation of boundary value analysis.

Automation of BVA

1. Choose distance functions based on input and output type.
    - Tip: Anything can be "stringfied", so Jaccard or even the difference in string length can be used

Automation of BVA

1. Choose distance functions based on input and output type.
    - Tip: Anything can be "stringfied", so Jaccard or even the difference in string length can be used
2. Define how to traverse the input space. Generate test input
    - Generation can be brute force (limited). Start at a value then do +1.

Automation of BVA

1. Choose distance functions based on input and output type.
    - Tip: Anything can be "stringfied", so Jaccard or even the difference in string length can be used
2. Define how to traverse the input space. Generate test input
    - Generation can be brute force (limited). Start at a value then do +1.
3. Give the input values to the program, and save output.
4. Calculate the program derivative (PD) of the input and output values.

Automation of BVA

1. Choose distance functions based on input and output type.
    - Tip: Anything can be "stringfied", so Jaccard or even the difference in string length can be used
2. Define how to traverse the input space. Generate test input
    - Generation can be brute force (limited). Start at a value then do +1.
3. Give the input values to the program, and save output.
4. Calculate the program derivative (PD) of the input and output values.
5. Save all inputs, outputs and corresponding PD in a table.
6. Filter the pairs (boundaries) with high program derivative.

What did we find in our BVA research?

We performed BVA on the Date class in Julia language

- Date constructor accepts 3 parameters as an input and returns a date in a string format.
  - `Date(2025, 11, 25) => "2025-11-25"`

What did we find in our BVA research?

We performed BVA on the Date class in Julia language

- Date constructor accepts 3 parameters as an input and returns a date in a string format.
  - `Date(2025, 11, 25) => "2025-11-25"`

Typemax: returns the maximum value of a specific type

- Similar to (Integer.MAX_VALUE) in Java

Typemax(Date) = "252522163911149-12-31"

What did we find in our BVA research?

We performed BVA on the Date class in Julia language

- Date constructor accepts 3 parameters as an input and returns a date in a string format.
  - `Date(2025, 11, 25) => "2025-11-25"`

Typemax: returns the maximum value of a specific type

- Similar to (Integer.MAX_VALUE) in Java

Typemax(Date) = "252522163911149-12-31"

What happens with typemax(Date) + Day(1)?

What did we find in our BVA research?

We performed BVA on the Date class in Julia language

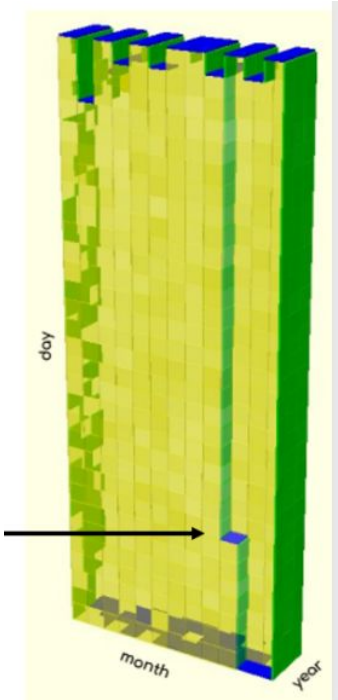
- Date constructor accepts 3 parameters as an input and returns a Date in a string format.
  - `Date(2025, 11, 25) => "2025-11-25"`

Typemax: returns the maximum value of a specific type

- Similar to (Integer.MAX_VALUE) in Java

Typemax(Date) = "252522163911149-12-31"

What happens with typemax(Date) + Day(1)?

- You get valid dates! Note that the actual max is October 7th!

Would the implementation match the tester's expectation?